Міністерство освіти і науки України

Національний технічний університет України

“Київський політехнічний інститут імені Ігоря Сікорського”

Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

Лабораторна робота №1

Методологія інженерії програмного забезпечення

Тема: Польові дослідження

Виконав:

студент групи ІП-51мн

Панченко С. В.

Перевірив:

Щебланін Ю. М.

Київ 2026

ЗМІСТ

1 Мета 3
2 Завдання 4
3 Виконання 5
3.1 Перевірка «Закону тривіальності» 5
3.2 Аналіз документації як «комунікаційного запобіжника» 7
3.3 Динаміка Code Review: Синдром «Пінг-Понгу» 8
4 Висновок 9
А Лістинг коду 10
А.1 main.py 10
А.2 fetch_prs.py 16
А.3 pr_classes.py 19

1 МЕТА

Ознайомитися з типами польових досліджень (Field studies, Field experiments).

2 ЗАВДАННЯ

1. Визначити мету дослідження та обрати тип (варіант) та метод дослідження. Погодити з викладачем.
2. Створити план дослідження та обрати або створити необхідні інструменти дослідження.
3. Виконати дослідження.
4. Аналізувати дані дослідження та зробити висновки.
5. Створити звіт (протокол та презентацію) за результатами дослідження за обраним варіантом завдання дослідження.

3 ВИКОНАННЯ

Об’єктом дослідження було обрано репозиторій відкритого коду `typst/typst` на платформі GitHub. Для виконання завдання застосовано тип дослідження “Польові дослідження” (Field Studies), зокрема методики третього ступеня (Third Degree): **Analysis of Electronic Databases of Work Performed** (аналіз метаданих понад 2000 Pull Requests) та **Documentation Analysis** (аналіз текстових описів артефактів).

3.1 Перевірка «Закону тривіальності»

На рисунках 3.1 та 3.2 зображено залежність між обсягом внесених змін (кількість доданих та видалених рядків) та щільністю обговорення (кількість коментарів на рядок коду).

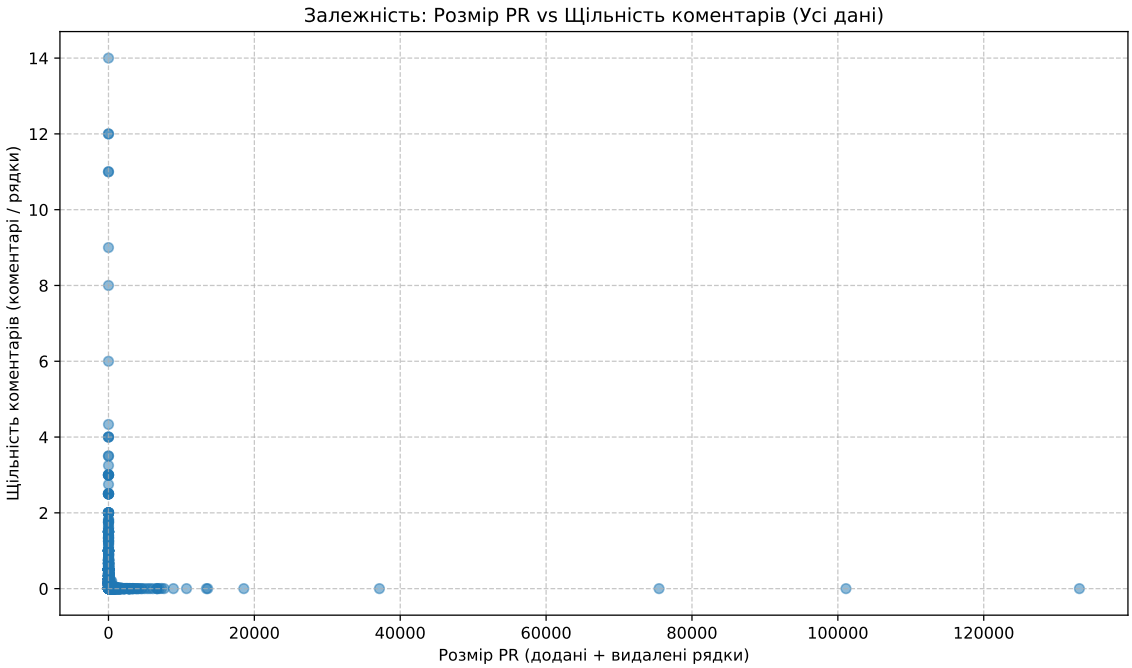

Рисунок 3.1 — Залежність розміру PR від щільності коментарів (всі дані)

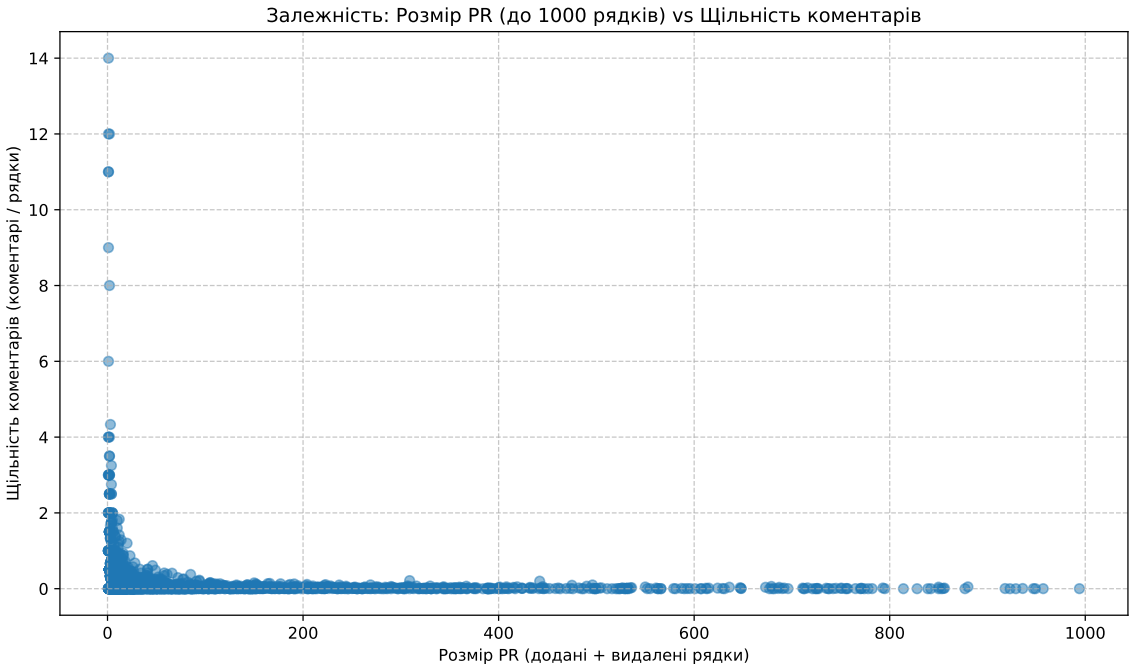

Рисунок 3.2 — Залежність розміру PR від щільності коментарів (до 1000 рядків)

Діаграма розсіювання демонструє класичну L-подібну залежність: дрібні PR (до 100 рядків) викликають непропорційно більшу кількість коментарів, тоді як великі рефакторинги (>1000 рядків) проходять рев'ю майже без обговорень.

Крім того, рисунок 3.3 демонструє відсутність лінійної масштабованості між кількістю змінених файлів та кількістю коментарів. PR, що зачіпають понад 40 файлів, мають вкрай низьку кількість коментарів, що вказує на виникнення «когнітивної стелі» у рев'юерів.

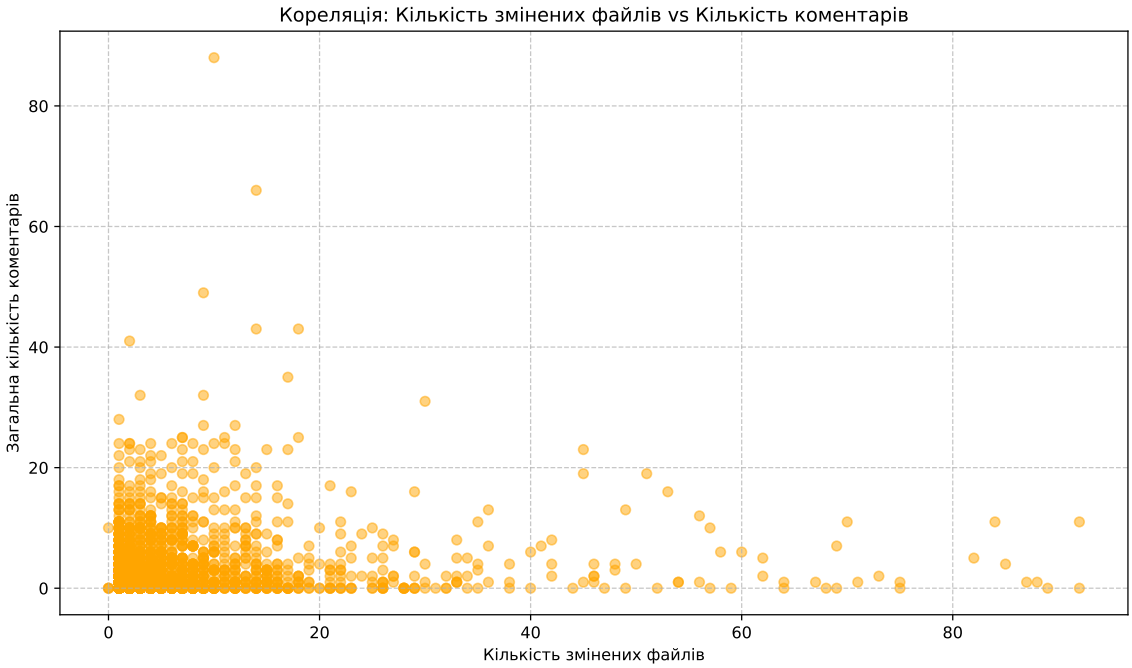

Рисунок 3.3 — Кореляція кількості змінених файлів та коментарів

3.2 Аналіз документації як «комунікаційного запобіжника»

Метою наступного етапу було дослідити вплив якості текстового опису Pull Request на комунікаційне навантаження. Результати наведено на рисунках 3.4 та 3.5.

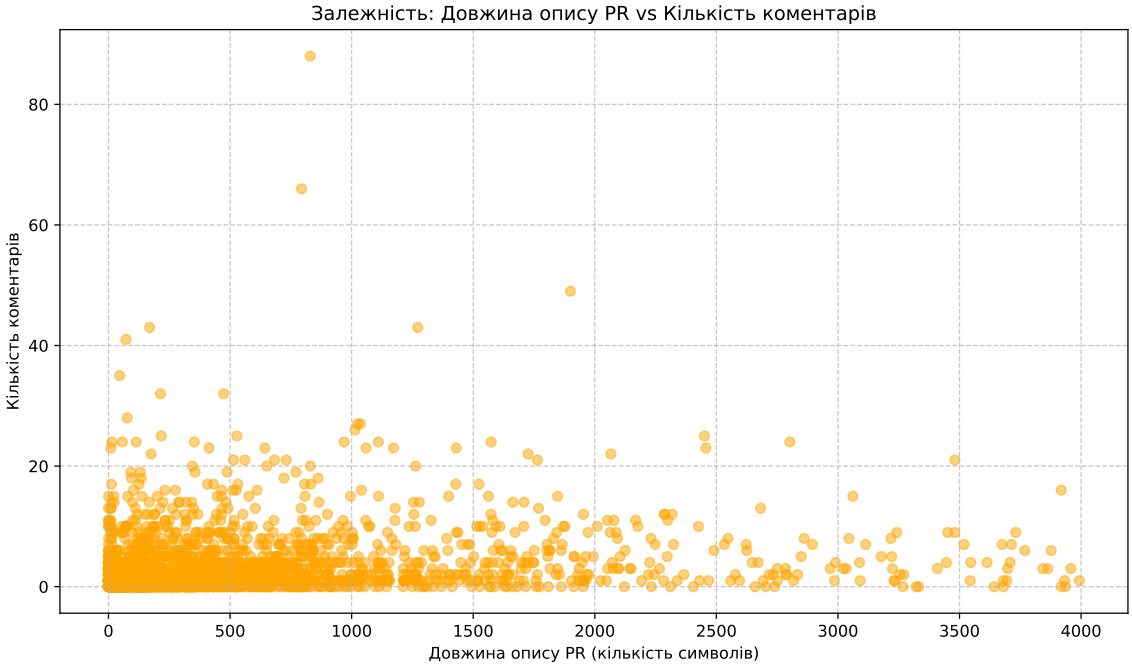

Рисунок 3.4 — Залежність кількості коментарів від довжини опису PR

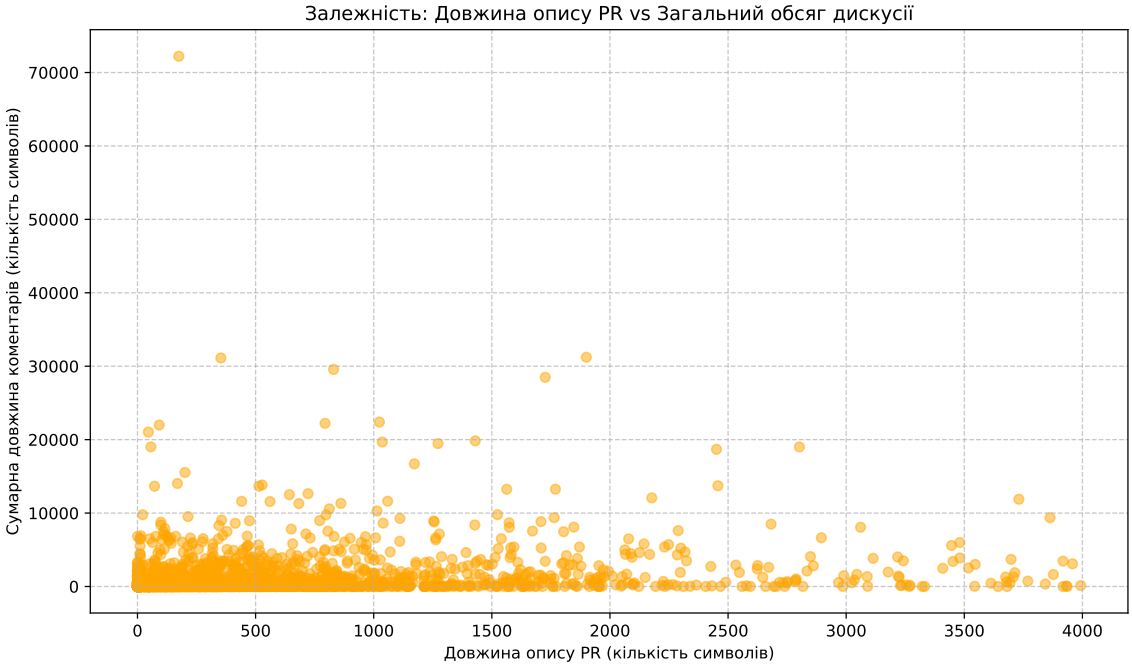

Рисунок 3.5 — Залежність загального обсягу дискусії від довжини опису PR

Спостерігається клиноподібний розподіл: PR із мінімальним описом (менше 500 символів) генерують найбільші комунікаційні сплески (аномалії до 70 000 символів у коментарях). Детальний опис локалізує обговорення, роблячи рев'ю більш передбачуваним.

3.3 Динаміка Code Review: Синдром «Пінг-Понгу»

На рисунку 3.6 проаналізовано вплив мікро-ітерацій (чергування коментаря рев'юера та дрібного коміту автора) на загальний час життєвого циклу Пул-реквесту.

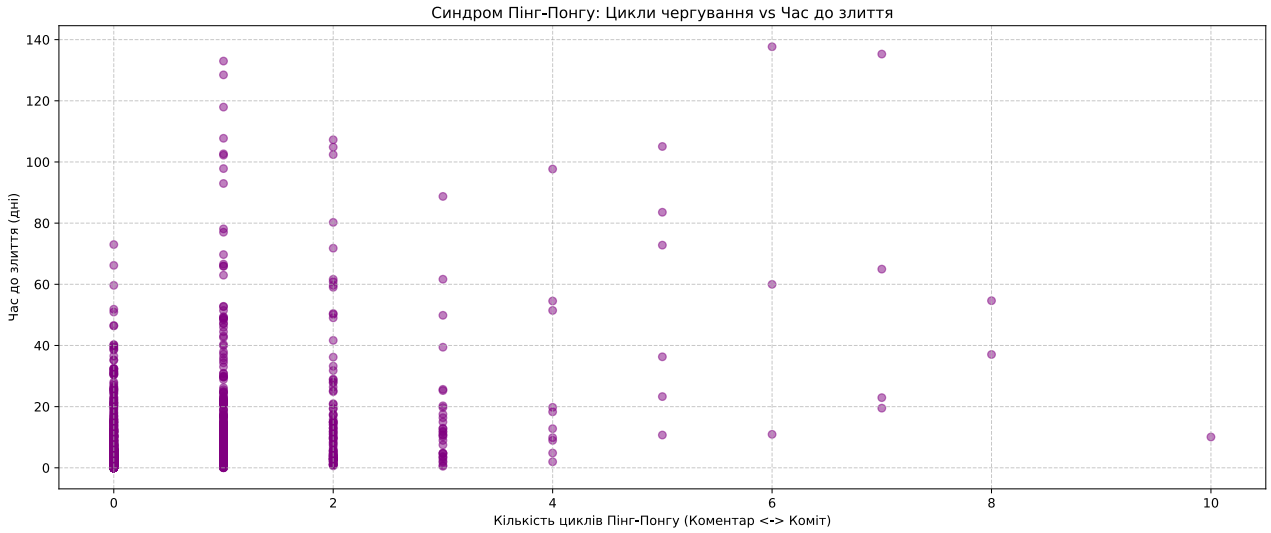

Рисунок 3.6 — Синдром Пінг-Понгу: вплив циклів чергування на час злиття

Зібрана статистика виявляє майже експоненціальне зростання часу інтеграції: у базовому сценарії (0 циклів) процес займає в середньому 1.7 днів. При наявності 6 і більше циклів комунікаційного тертя розробка паралізується, і час до злиття сягає понад 30 днів.

4 ВИСНОВОК

У ході польового дослідження екосистеми typst/typst доведено, що швидкість розробки суттєво залежить від соціально-комунікаційних патернів. Використання методів Analysis of Electronic Databases та Documentation Analysis дозволило підтвердити закон тривіальності: спільнота схильна до мікроменеджменту дрібних правок, уникаючи глибокого аналізу масштабних змін через когнітивне перевантаження.

Дослідження також підтвердило критичну важливість якісної документації артефактів. Відсутність початкового контексту в PR провокує тривалі дискусії, тоді як детальний опис знижує їхній обсяг у 3-5 разів. Водночас виявлено деструктивний вплив синдрому «Пінг-Понгу»: ітеративний формат спілкування під час Code Review є вкрай неефективним і здатен збільшити час прийняття рішень до 17 разів.

Для оптимізації процесів розробки команді рекомендується впровадити обов’язкові шаблони описів коду та перейти до пакетного рецензування, що дозволить мінімізувати накладні витрати на постійну зміну контексту.

Додаток А

Лістинг коду

А.1 main.py

```
#!/usr/bin/env python
# coding: utf-8

import typing
import pickle
import os
import pathlib
import datetime
import numpy as np
import matplotlib.pyplot as plt
import scipy.stats

import pr_classes

def main():
    # Налаштування шляхів
    SCRIPT_PATH = pathlib.Path(os.path.abspath(__file__)) # або вкажіть
ваш шлях явно
    SCRIPT_DIR = SCRIPT_PATH.parent
    BUILD_DIR = SCRIPT_DIR / 'build'
    PLOTS_DIR = BUILD_DIR / 'plots'

    # Створюємо папку для графіків, якщо її немає
    PLOTS_DIR.mkdir(parents=True, exist_ok=True)

    # Завантаження даних
    with open(BUILD_DIR / 'ser_prs.pickle', 'rb') as fileio:
        ser_prs = typing.cast(pr_classes.PrBlob, pickle.load(fileio))

    #
==========================================================================
    # Графік 1: Розмір PR vs Щільність (Без фільтрів)
    #
==========================================================================
    size_list_unfiltered: list[int] = []
```

```
    density_list_unfiltered: list[float] = []

    for pr in ser_prs.prs:
        size = pr.additions + pr.deletions
        if size == 0:
            continue
        density = len(pr.comments) / size
        size_list_unfiltered.append(size)
        density_list_unfiltered.append(density)

    plt.figure(figsize=(10, 6))
    plt.scatter(size_list_unfiltered, density_list_unfiltered,
alpha=0.5)
    plt.title('Залежність: Розмір PR vs Щільність коментарів (Усі
дані)')
    plt.xlabel('Розмір PR (додані + видалені рядки)')
    plt.ylabel('Щільність коментарів (коментарі / рядки)')
    plt.grid(True, linestyle='--', alpha=0.7)

    plt.tight_layout()
    plt.savefig(PLOTS_DIR / '1_size_vs_density_unfiltered.svg')
    plt.close()

    #
==========================================================================
    # Графік 2: Розмір PR vs Щільність (Відфільтровано до 1000 рядків)
    #
==========================================================================
    size_list: list[int] = []
    density_list: list[float] = []

    for pr in ser_prs.prs:
        size = pr.additions + pr.deletions
        if size == 0 or size > 1000:
            continue
        density = len(pr.comments) / size
        size_list.append(size)
        density_list.append(density)
```

```
    corr_size_density, p_value1 = scipy.stats.pearsonr(size_list,
density_list)
    print(f"Кореляція (Розмір vs Щільність): {corr_size_density:.4f}
(p-value: {p_value1:.4e})")

    plt.figure(figsize=(10, 6))
    plt.scatter(size_list, density_list, alpha=0.5)
    plt.title('Залежність: Розмір PR (до 1000 рядків) vs Щільність
коментарів')
    plt.xlabel('Розмір PR (додані + видалені рядки)')
    plt.ylabel('Щільність коментарів (коментарі / рядки)')
    plt.grid(True, linestyle='--', alpha=0.7)

    plt.tight_layout()
    plt.savefig(PLOTS_DIR / '2_size_vs_density_filtered.svg')
    plt.close()

    #
==============================================================================
    # Графік 3: Кількість змінених файлів vs Кількість коментарів
    #
==============================================================================
    files_list: list[int] = []
    comments_count_list: list[int] = []

    for pr in ser_prs.prs:
        if pr.changedFiles < 100:
            files_list.append(pr.changedFiles)
            comments_count_list.append(len(pr.comments))

    corr_files_comments, p_value2 = scipy.stats.pearsonr(files_list,
comments_count_list)
    print(f"Кореляція (Файли vs Коментарі): {corr_files_comments:.4f}
(p-value: {p_value2:.4e})")

    plt.figure(figsize=(10, 6))
    plt.scatter(files_list, comments_count_list, alpha=0.5,
```

```
color='orange')
    plt.title('Кореляція: Кількість змінених файлів vs Кількість
коментарів')
    plt.xlabel('Кількість змінених файлів')
    plt.ylabel('Загальна кількість коментарів')
    plt.grid(True, linestyle='--', alpha=0.7)

    plt.tight_layout()
    plt.savefig(PLOTS_DIR / '3_files_vs_comments.svg')
    plt.close()

    #
==============================================================================
    # Графік 4: Довжина опису PR vs Кількість коментарів
    #
==============================================================================
    pr_body_lens: list[int] = []
    pr_comments_counts: list[int] = []

    for pr in ser_prs.prs:
        body_text = pr.body if hasattr(pr, 'body') and pr.body else ""
        body_len = len(body_text)
        if body_len < 4000:
            pr_body_lens.append(body_len)
            pr_comments_counts.append(len(pr.comments))

    plt.figure(figsize=(10, 6))
    plt.scatter(pr_body_lens, pr_comments_counts, alpha=0.5,
color='orange')
    plt.title('Залежність: Довжина опису PR vs Кількість коментарів')
    plt.xlabel('Довжина опису PR (кількість символів)')
    plt.ylabel('Кількість коментарів')
    plt.grid(True, linestyle='--', alpha=0.7)

    plt.tight_layout()
    plt.savefig(PLOTS_DIR / '4_body_len_vs_comment_count.svg')
    plt.close()
```

```
    #
==============================================================================
    # Графік 5: Довжина опису PR vs Загальний обсяг дискусії
    #
==============================================================================
    pr_body_lens_total: list[int] = []
    pr_comments_len_total: list[int] = []

    for pr in ser_prs.prs:
        body_text = pr.body if hasattr(pr, 'body') and pr.body else ""
        body_len = len(body_text)
        if body_len < 4000:
            pr_body_lens_total.append(body_len)
            # Сумуємо довжину тексту всіх коментарів (з перевіркою на
None)
            total_len = sum(len(c.body) for c in pr.comments if
hasattr(c, 'body') and c.body)
            pr_comments_len_total.append(total_len)

    plt.figure(figsize=(10, 6))
    plt.scatter(pr_body_lens_total, pr_comments_len_total, alpha=0.5,
color='orange')
    plt.title('Залежність: Довжина опису PR vs Загальний обсяг
дискусії')
    plt.xlabel('Довжина опису PR (кількість символів)')
    plt.ylabel('Сумарна довжина коментарів (кількість символів)')
    plt.grid(True, linestyle='--', alpha=0.7)

    plt.tight_layout()
    plt.savefig(PLOTS_DIR / '5_body_len_vs_total_comment_len.svg')
    plt.close()

    #
==============================================================================
    # Графік 6: Синдром Пінг-Понгу
    #
==============================================================================
    class CommitEvent(typing.NamedTuple):
```

```
        dt: datetime.datetime

    class CommentEvent(typing.NamedTuple):
        dt: datetime.datetime

    type PrEvent = CommitEvent | CommentEvent

    ping_pong_counts: list[int] = []
    merge_times_days: list[float] = []

    for pr in ser_prs.prs:
        if pr.mergedAt is None:
            continue

        events: list[PrEvent] = []
        for c in pr.commits:
events.append(CommitEvent(datetime.datetime.fromisoformat(c.committedDate.repl
'+00:00'))))
        for c in pr.comments:
            if c.author.login != pr.author.login:
events.append(CommentEvent(datetime.datetime.fromisoformat(c.createdAt.replace
'+00:00'))))

        events.sort(key=lambda x: x.dt)

        first_comment_idx = -1
        for i, e in enumerate(events):
            if isinstance(e, CommentEvent):
                first_comment_idx = i
                break

        if first_comment_idx == -1:
            continue

        ping_pongs = 0
        prev_event = events[first_comment_idx]
        for e in events[first_comment_idx + 1:]:
```

```
            if e.__class__ != prev_event.__class__:
                ping_pongs += 1
                prev_event = e

        cycles = ping_pongs // 2

        created =
datetime.datetime.fromisoformat(pr.createdAt.replace('Z', '+00:00'))
        merged =
datetime.datetime.fromisoformat(pr.mergedAt.replace('Z', '+00:00'))
        duration_days = (merged - created).total_seconds() / (3600 *
24)

        if duration_days > 150:
            continue

        ping_pong_counts.append(cycles)
        merge_times_days.append(duration_days)

    plt.figure(figsize=(14, 6))
    plt.scatter(ping_pong_counts, merge_times_days, alpha=0.5,
color='purple')
    plt.title('Синдром Пінг-Понгу: Цикли чергування vs Час до злиття')
    plt.xlabel('Кількість циклів Пінг-Понгу (Коментар <-> Коміт)')
    plt.ylabel('Час до злиття (дні)')
    plt.grid(True, linestyle='--', alpha=0.7)

    plt.tight_layout()
    plt.savefig(PLOTS_DIR / '6_ping_pong_syndrome.svg')
    plt.close()

if __name__ == '__main__':
    main()
```

A.2 fetch_prs.py

```
import typing
import pickle
import os
import pathlib
```

```python
import shutil
import argparse
import asyncio
import json
import pr_classes


SCRIPT_PATH = pathlib.Path(os.path.abspath(__file__))
SCRIPT_DIR = SCRIPT_PATH.parent
BUILD_DIR = SCRIPT_DIR / 'build'

def needs_rebuild(output_path: pathlib.Path | str, input_paths:
typing.Iterable[pathlib.Path | str]) -> bool:
    if not os.path.exists(output_path):
        res = True
    else:
        output_stat = os.stat(output_path)
        res = any(os.stat(input_path).st_mtime > output_stat.st_mtime
for input_path in input_paths)
    if res:
        print(f'build: {output_path}')
    return res

def clean_command():
    shutil.rmtree(BUILD_DIR, ignore_errors=True)

async def main():
    parser = argparse.ArgumentParser()
    subparsers = parser.add_subparsers(dest='subcommand')
    parser_clean = subparsers.add_parser('clean')
    parser_clean.set_defaults(func=clean_command)
    args = parser.parse_args()
    if hasattr(args, 'func'):
        args.func()
        return

    BUILD_DIR.mkdir(exist_ok=True)
    target_dir = SCRIPT_DIR.parent.parent / 'typst'
```

```
    pr_list_path = BUILD_DIR / 'pr_list.json'
    if needs_rebuild(pr_list_path, ()):
    # if True:
        cmd = ("gh", "pr", "list", "--state", "all", "--limit", "5000",
"--json", "number")
        print(f'{cmd=}')
        process = await asyncio.create_subprocess_exec(*cmd,
cwd=target_dir, stdout=asyncio.subprocess.PIPE,
stderr=asyncio.subprocess.PIPE)
        stdout, stderr = await process.communicate()
        assert process.returncode == 0
        stdout = stdout.decode()
        with open(pr_list_path, 'w') as fileio:
            fileio.write(stdout)
        pr_list = json.loads(stdout)
    else:
        with open(pr_list_path) as fileio:
            pr_list = json.load(fileio)

    import itertools
    pr_numbers = tuple(int(pr["number"]) for pr in pr_list)
    pr_paths = tuple(BUILD_DIR / f'pr_view_{number}.json' for number in
pr_numbers)
    pr_rebuilds = ((number, path) for number, path in zip(pr_numbers,
pr_paths) if needs_rebuild(path, ()))

    is_pulled_new_prs = False
    for pr_batch in itertools.batched(pr_rebuilds, 100):
        async with asyncio.TaskGroup() as tg:
            async def tg_task(number: int, pr_path: pathlib.Path):
                nonlocal is_pulled_new_prs
                cmd = ("gh", "pr", "view", str(number), "--json",
"body,number,title,author,additions,deletions,changedFiles,createdAt,mergedAt,
                print(f'{cmd=}')
                process = await
asyncio.subprocess.create_subprocess_exec(*cmd, cwd=target_dir,
stdout=asyncio.subprocess.PIPE, stderr=asyncio.subprocess.PIPE,)
                stdout, stderr = await process.communicate()
```

```
                if process.returncode != 0:
                    print(f'pr {number} failed with error:
{process.returncode}, {stderr.decode()=}')
                else:
                    stdout = stdout.decode()
                    with open(pr_path, 'w') as fileio:
                        fileio.write(stdout)
                    is_pulled_new_prs = True

            for i, pr in pr_batch:
                tg.create_task(tg_task(i, pr))
        await asyncio.sleep(0.5)

    # if is_pulled_new_prs:
    if True:
        ser_prs_list: list[pr_classes.PullRequest] = []
        for pr in pr_paths:
            with open(pr) as fileio:
                data = json.load(fileio)

ser_prs_list.append(pr_classes.PullRequest.deserialize(data))
        ser_prs = pr_classes.PrBlob(tuple(ser_prs_list))

        with open(BUILD_DIR / 'ser_prs.pickle', 'wb') as fileio:
            pickle.dump(ser_prs, fileio)

    with open(BUILD_DIR / 'ser_prs.pickle', 'rb') as fileio:
        ser_prs = pickle.load(fileio)
        assert isinstance(ser_prs, pr_classes.PrBlob)

if __name__ == '__main__':
    asyncio.run(main())
```

A.3 pr_classes.py

```
import typing


class Comment(typing.NamedTuple):
    class Author(typing.NamedTuple):
```

```
        login: str

    id: str
    author: Author
    authorAssociation: str
    body: str
    createdAt: str
    includesCreatedEdit: bool
    isMinimized: bool
    minimizedReason: str
    reactionGroups: list[typing.Any]
    url: str
    viewerDidAuthor: bool

    @classmethod
    def deserialize(cls, data: dict[typing.Any, typing.Any]) ->
typing.Self:
        return cls(
            id=data['id'],
            author=cls.Author(**data['author']),
            authorAssociation=data['authorAssociation'],
            body=data['body'],
            createdAt=data['createdAt'],
            includesCreatedEdit=data['includesCreatedEdit'],
            isMinimized=data['isMinimized'],
            minimizedReason=data['minimizedReason'],
            reactionGroups=data['reactionGroups'],
            url=data['url'],
            viewerDidAuthor=data['viewerDidAuthor'],
        )

class Commit(typing.NamedTuple):
    class Author(typing.NamedTuple):
        email: str
        id: str
        login: str
        name: str
```

```
    authoredDate: str
    authors: tuple[Author, ...]
    committedDate: str
    messageBody: str
    messageHeadline: str
    oid: str

    @classmethod
    def deserialize(cls, data: dict[typing.Any, typing.Any]) ->
typing.Self:
        return cls(
            authoredDate=data['authoredDate'],
            authors=tuple(cls.Author(**c) for c in data['authors']),
            committedDate=data['committedDate'],
            messageBody=data['messageBody'],
            messageHeadline=data['messageHeadline'],
            oid=data['oid'],
        )
# ({'id': 'PRR_kwDODI8QW87kNtEP', 'author': {'login': 'isuffix'},
'authorAssociation': 'COLLABORATOR', 'body': '', 'submittedAt':
'2026-02-19T22:09:17Z', 'includesCreatedEdit': False, 'reactionGroups':
[], 'state': 'COMMENTED', 'commit': {'oid':
'cab0a5b784a21ffd4f01b99d3caf76511ea3badd'}},)
# [{'content': 'THUMBS_UP', 'users': {'totalCount': 2}}, {'content':
'HEART', 'users': {'totalCount': 2}}]

class ReviewReactionGroupUsers(typing.NamedTuple):
    totalCount: int

class ReviewReactionGroup(typing.NamedTuple):
    content: str
    users: ReviewReactionGroupUsers

    @classmethod
    def deserialize(cls, data: dict[typing.Any, typing.Any]) ->
typing.Self:
        return cls(
            content=data['content'],
```

```
            users=ReviewReactionGroupUsers(**data['users']),
        )

class ReviewAuthor(typing.NamedTuple):
    login: str

class ReviewCommit(typing.NamedTuple):
    oid: str

class Review(typing.NamedTuple):
    id: str
    author: ReviewAuthor
    authorAssociation: str
    body: str
    submittedAt: str
    includesCreatedEdit: bool
    reactionGroups: tuple[ReviewReactionGroup, ...]
    state: str
    commit: ReviewCommit

    @classmethod
    def deserialize(cls, data: dict[typing.Any, typing.Any]) ->
typing.Self:
        return cls(
            id=data['id'],
            author=ReviewAuthor(**data['author']),
            authorAssociation=data['authorAssociation'],
            body=data['body'],
            submittedAt=data['submittedAt'],
            includesCreatedEdit=data['includesCreatedEdit'],
            reactionGroups=tuple(ReviewReactionGroup.deserialize(c) for
c in data['reactionGroups']),
            state=data['state'],
            commit=ReviewCommit(**data['commit']),
        )

# ({'id': 'LA_kwDODI8QW88AAAABPfG87A', 'name': 'math', 'description':
'Related to math syntax, layout, etc.', 'color': '007AFF'}, {'id':
```

```
'LA_kwDODI8QW88AAAAB8X2QTw', 'name': 'interface', 'description': "PRs
that add to or change Typst's user-facing interface as opposed to
internals or docs changes.", 'color': 'a4ebe7'})

class PrLabel(typing.NamedTuple):
    id: str
    name: str
    description: str
    color: str

class PullRequest(typing.NamedTuple):
    class Author(typing.NamedTuple):
        login: str
        id: str | None
        name: str | None
        is_bot: bool

        @classmethod
        def deserialize(cls, data: dict[typing.Any, typing.Any]) ->
typing.Self:
            return cls(
                login=data['login'],
                id=data.get('id'),
                name=data.get('name'),
                is_bot=data['is_bot'],
            )

    additions: int
    author: Author
    changedFiles: int
    closedAt: str
    comments: tuple[Comment, ...]
    commits: tuple[Commit, ...]
    createdAt: str
    deletions: int
    labels: tuple[PrLabel, ...]
    mergedAt: str | None
    number: int
```

```
    reviewDecision: str
    reviews: tuple[Review, ...]
    title: str
    body: str

    @classmethod
    def deserialize(cls, data: dict[typing.Any, typing.Any]) ->
typing.Self:
        return cls(
            additions=data['additions'],
            body=data['body'],
            author=cls.Author.deserialize(data['author']),
            changedFiles=data['changedFiles'],
            closedAt=data['closedAt'],
            comments=tuple(Comment.deserialize(c) for c in
data['comments']),
            commits=tuple(Commit.deserialize(c) for c in
data['commits']),
            createdAt=data['createdAt'],
            deletions=data['deletions'],
            labels=tuple(PrLabel(**c) for c in data['labels']),
            mergedAt=data['mergedAt'],
            number=data['number'],
            reviewDecision=data['reviewDecision'],
            reviews=tuple(Review.deserialize(c) for c in
data['reviews']),
            title=data['title'],
        )

class PrBlob(typing.NamedTuple):
    prs: tuple[PullRequest, ...]
```